

ComFree-TDMPC: Making the Loop Fast Enough to Ask the Question

A throughput fix, then a four-rung ablation of distillation, system identification and terminal value learning on in-hand cube reorientation, and two further runs chasing the policy that never catches the planner.

What this project is

A robot hand has to turn a cube to a target orientation. The controller is **MPPI**: at every control step it imagines a few hundred different action sequences, simulates all of them, and executes the first action of the best one. What makes this project unusual is that the simulator it imagines with is a **real differentiable physics engine** (ComFree-Sim), not a neural network trained to imitate physics. Everything TD-MPC normally learns about dynamics is already known.

That leaves three things worth learning, and they are exactly the three things this report switches on one at a time:

Component	What it is meant to do
Distillation	The planner simulates thousands of trajectories per step and throws almost all of them away. Keep the best ones as training data.
System identification (SysID)	The planner's simulator has the wrong physical numbers. Fit the 71 unknown parameters from what actually happened on the robot.
Terminal Q	The planner can only look 8 steps ahead. A learned value function tells it what the state at step 8 is worth, extending its horizon for free.

The headline result

	Before	After	Change
Environment steps collected per second	9.2	55.7	6.1x
Training transitions produced per second	587	3567	6.1x
Environment steps in one hour	33k	201k	6.1x

The GPU was doing one thing at a time when it could do sixteen. Fixing that multiplied the data rate by 6.1x on the same card, without changing the algorithm.

Part 1 -- Why the loop was slow

The first thing to measure was where the time went. The answer was blunt: **98% of the wall clock was the physics simulator**, and the simulator was being asked to do far less work than it could.

A planning step is deep, but narrow

One MPPI planning step simulates a horizon of H steps, and those H steps must happen in order -- step 2 needs the state produced by step 1. That is the *depth*, and it cannot be parallelised. But all the sampled action sequences within one step are independent. That is the *width*, and a GPU eats width for free.

Measured on the RTX 4080: stepping **256 worlds** takes **3.3 ms**. Stepping **4096 worlds** -- sixteen times as many -- takes **10.0 ms**, only 3.0 times longer. The device was idle in the width direction.

The original code ran **one** environment, and that environment's planner used 256 worlds. So all the spare width was wasted. The fix is to run many environments as extra worlds in the same bank: environment e , sample i becomes world $e \times N + i$. Sixteen environments then plan in the same kernel launches that one used to.

Throughput: the planner is deep, not wide -- so fill the width

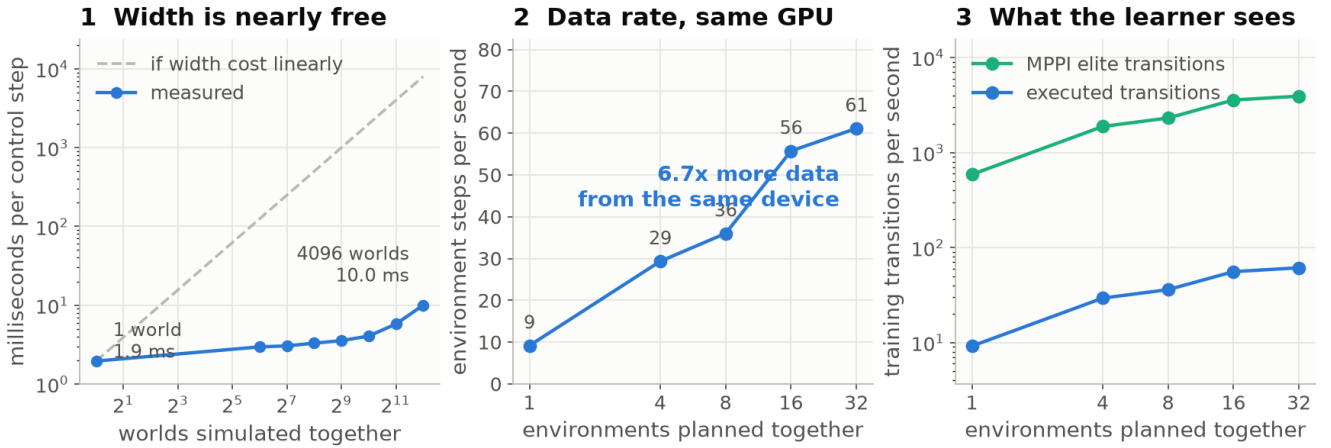


Figure 1. Left: the cost of one control step against how many worlds are stepped together -- the dashed line is what it would cost if width were not free. Middle: environment steps collected per second against how many environments plan together. Right: the two kinds of training data this produces, on a log scale.

Table 1. Simulator cost against width.

Worlds stepped together	ms per control step	Physics steps / s
1	1.95	5.13e+03
64	2.95	2.17e+05
128	3.04	4.21e+05
256	3.29	7.78e+05
512	3.54	1.45e+06
1024	4.04	2.53e+06
2048	5.75	3.56e+06
4096	9.98	4.10e+06

Was the learner the bottleneck instead? No.

One gradient update of the whole agent (encoder, an ensemble of 5 Q networks, the policy) costs **4.6 ms**. At the original data rate of 9.2 environment steps per second, the learner was idle roughly 95% of the time. Speeding up data collection therefore buys gradient steps at almost no extra cost, and this report spends that budget: the

ablation runs 1.5 gradient updates per environment step.

Part 2 -- Checking that the fast version is the same algorithm

A speedup that quietly changes the maths is not a speedup. But the obvious test -- "do the two versions produce identical trajectories?" -- gives a misleading answer here, and it is worth saying why.

The physics engine packs the contacts of *all* worlds into one shared pool. Change how many worlds there are, and float32 numbers get added in a different order. That is a rounding difference of about 1 part in 10 million at the first step. Contact dynamics are chaotic, so that difference grows by roughly a factor of ten per control step. After 40 steps two mathematically identical simulations visibly disagree -- **including the original one compared against itself**.

So every check below reports two numbers: the disagreement being tested, and a **noise floor** measured by running the reference against a re-run of itself. A check passes when the first is no worse than the second. The structural question -- does world $e \times N + i$ really carry environment e 's state, goal and action? -- is answered separately at a single step, where chaos has had no time to amplify anything and agreement must be at rounding level.

Check	Result	Pass
vec env reproduces single env	step 1: 3.0e-08 (floor 1.2e-10) step 40: 2.4e-06 (floor 1.8e-05)	yes
batched rollout matches single rollout	H=1: 6.0e-05 (floor 5.0e-05) H=8: 5.0e-02 (floor 3.6e-02)	yes
each env is scored against its own goal	max dvalue /scale=8.8e-03 (goals differ by 39.4)	yes
each env receives its own action	max dqpos =1.2e-07 (actions differ by 1.8e-01)	yes
lanes do not interact	other lanes moved 3.0e-03 (floor 4.5e-03), lane 0 moved 2.0e-01	yes
buffer chunks are contiguous	same lane=True consecutive t=True shape=True	yes

Table 2. Correctness checks on the vectorised stack. The single-step figures (3e-08 for state, 1e-07 for actions) are at float32 rounding level, which is what shows the world layout is exactly right.

Cross-lane leakage measured 3.0e-03 while the noise floor of an identical re-run was 4.5e-03. The environments do not influence each other beyond arithmetic round-off.

Part 3 -- Buying gradient steps with planner configuration

Every environment step pays for one planning call, so the planner's settings set the budget for everything else. The question is not which setting plans best, but which plans best *per second*. Nine configurations were scored with no learning attached, on the true physics, 16 episodes at a time.

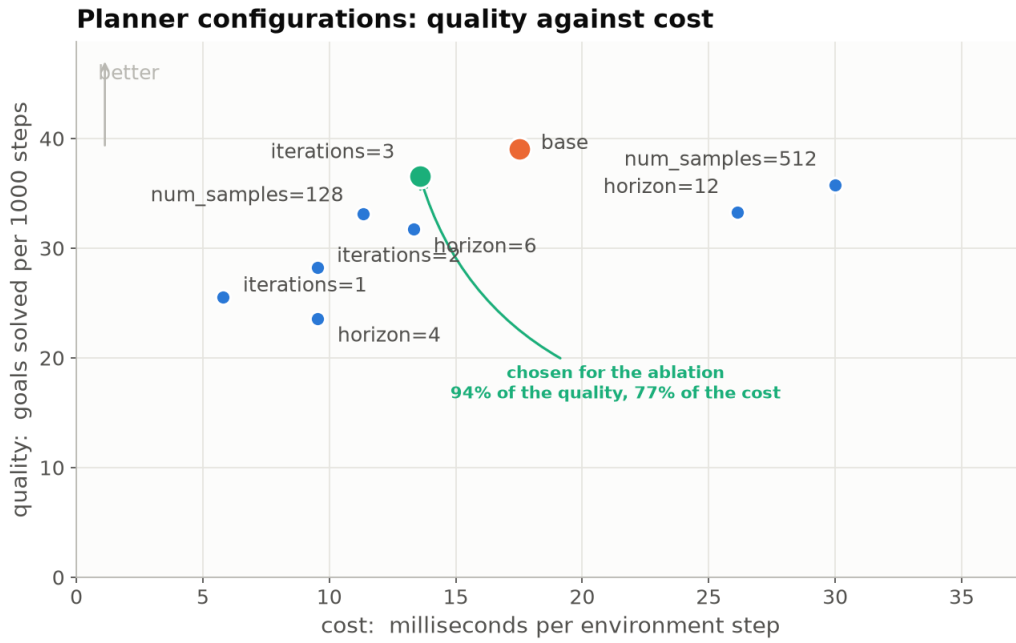


Figure 2. Each dot is one planner configuration. Up is better quality; left is cheaper. The base setting is orange, the setting chosen for the ablation is green.

Configuration	Goals / 1000 steps	ms / env step	Env steps / s
base	39.0	17.5	57
iterations=1	25.6	5.8	172
iterations=2	28.3	9.5	105
iterations=3	36.6	13.6	74
num_samples=128	33.1	11.3	88
num_samples=512	35.7	30.0	33
horizon=4	23.6	9.5	105
horizon=6	31.7	13.3	75
horizon=12	33.2	26.1	38

Table 3. Planner sweep. Three MPPI iterations instead of four keeps 94% of the quality for 77% of the cost; the saving was spent on gradient updates.

Part 4 -- The ablation

Six runs, each 150,000 environment steps, identical in everything except one switch per rung. Rungs A to D are the ladder proper -- one component added at a time; E and F vary only how the policy is fitted, on top of D. All of them face the same wrong physics: the simulator the planner uses is the model as compiled, while "reality" has 71 physical parameters displaced from it.

	Setting
Task	Allegro hand, in-hand cube reorientation about the palm normal
Environments in parallel	16
Planner	MPPI, horizon 8, 256 samples, 3 iterations, 32 elites
Worlds simulated together	16 x 256 = 4096
Episode length	120 control steps (2.4 s)
Environment steps per run	150,000
Gradient updates per env step	1.5
Physics gap	71 unknown parameters, displaced up to 0.25 in normalised units

Table 4. Shared experimental setup.

Training behaviour of the closed loop (the planner is driving)

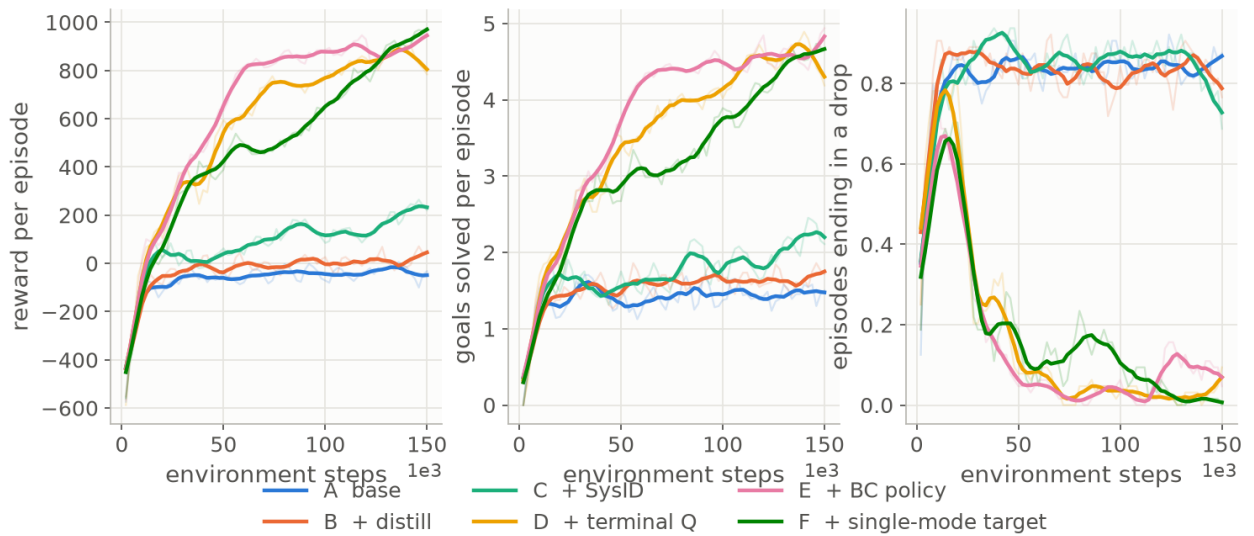


Figure 3. What the closed loop does during training. The planner is driving in all three panels, so these curves describe the planner's behaviour, not the learned policy's.

The distillation gap

The most important comparison in the whole report is Figure 4: the same episodes, the same axes, run once by the planner and once by the learned policy alone.

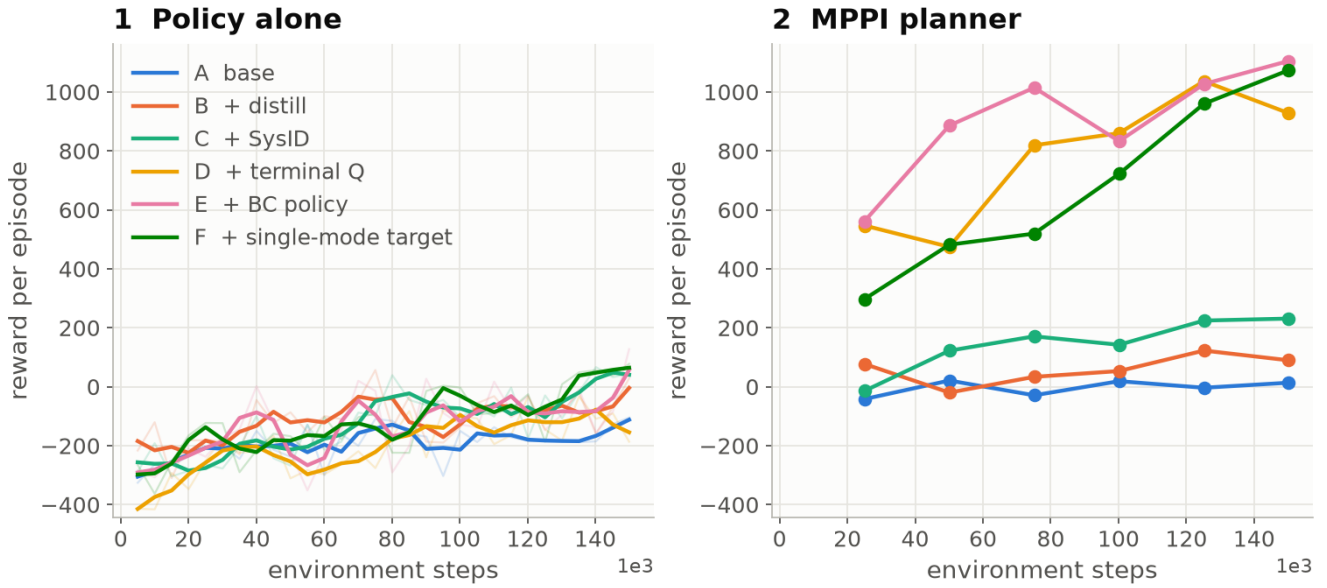


Figure 4. Left: the learned policy acting on its own, with no planning. Right: the MPPI planner. Both panels share one vertical scale, because the distance between them is the point.

Final performance of every rung on the ladder

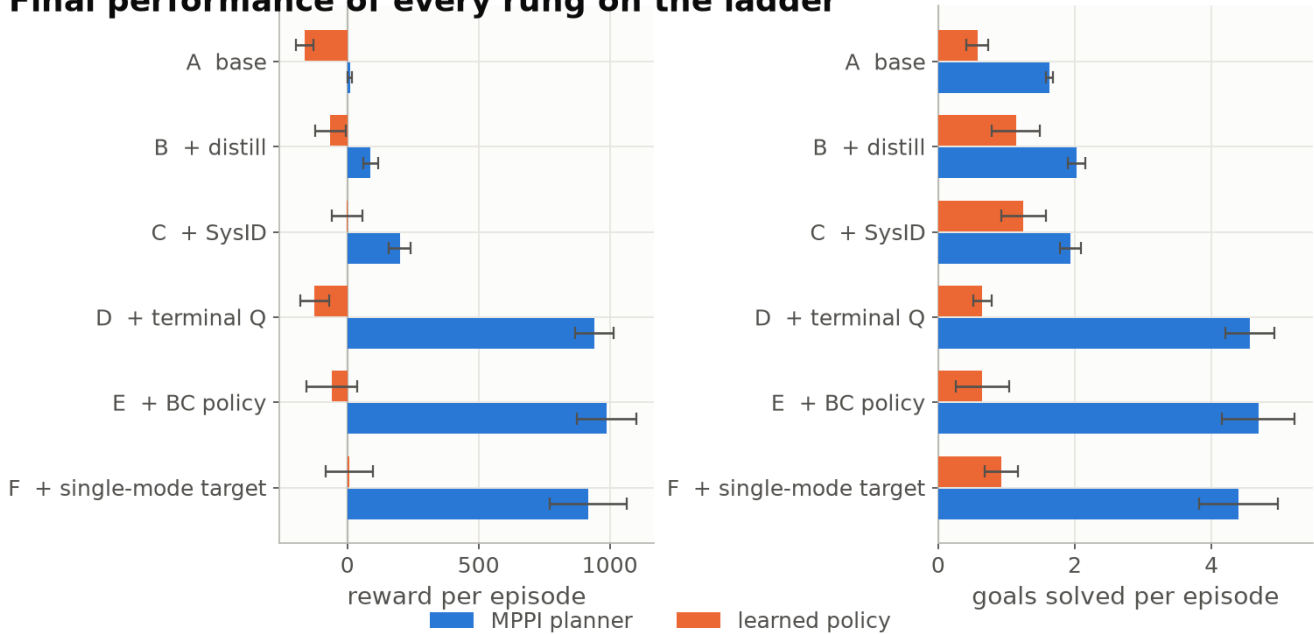


Figure 5. Final measured performance of every rung, planner against policy.

Run	Planner reward	Planner goals	Policy reward	Policy goals	theta err
A base	10 +/- 9	1.62 +/- 0.05	-161 +/- 33	0.57 +/- 0.16	0.098
B + distill	89 +/- 28	2.02 +/- 0.13	-65 +/- 59	1.14 +/- 0.36	0.098
C + SysID	199 +/- 40	1.94 +/- 0.15	-2 +/- 58	1.25 +/- 0.32	0.089
D + terminal Q	942 +/- 73	4.56 +/- 0.36	-124 +/- 54	0.65 +/- 0.13	0.084
E + BC policy	988 +/- 114	4.69 +/- 0.53	-60 +/- 97	0.65 +/- 0.39	0.110
F + single-mode target	918 +/- 146	4.40 +/- 0.58	7 +/- 89	0.93 +/- 0.24	0.096

Table 5. Each rung, averaged over the last evaluations of the run (last 3 for the planner, last 6 for the policy, 16 episodes each) with the spread across those evaluations. Reward is per episode; goals is how many target orientations were reached and held within one episode.

What each switch bought

Consecutive rungs differ in exactly one component, so the difference between two consecutive rows is what that component is worth. Rung F is not part of the ladder -- it is the change suggested by the diagnosis in Part 6, and it is read against rung E.

Switch added	Policy goals	change	Planner goals	change
A base	0.57	--	1.62	--
B + distill	1.14	+0.56	2.02	+0.40
C + SysID	1.25	+0.11	1.94	-0.08
D + terminal Q	0.65	-0.60	4.56	+2.62
E + BC policy	0.65	+0.00	4.69	+0.12
F + single-mode target	0.93	+0.28	4.40	-0.29

Table 6. Goals solved per episode, and the change each switch made relative to the rung before it.

Part 5 -- System identification

The planner's simulator starts with the wrong physical numbers -- 71 of them: joint damping, rotor inertia, dry friction, servo gains, contact friction per finger, link masses, the cube's mass and inertia, and the two constants of the contact model. SysID fits them from short chunks of real trajectory replayed inside the simulator.

The trick that makes it cheap is the same one that made the planner fast: every probe direction and every data chunk is a separate world, so one descent iteration is two batched rollouts no matter how many parameters there are.

System identification: 71 unknown parameters, fitted from replayed chunks

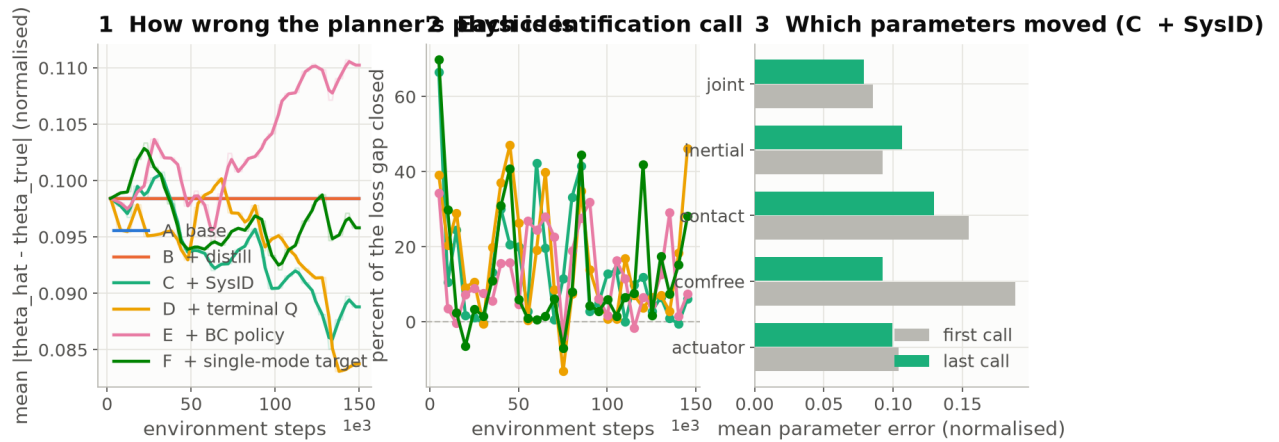


Figure 6. Left: how wrong the planner's physics is over training. Middle: the fraction of the achievable loss gap each identification call closes. Right: which groups of parameters actually moved.

Parameter group	How many	Error at first fit	At last fit	Change
actuator	8	0.1037	0.0994	-4%
comfree	2	0.1879	0.0921	-51%
contact	7	0.1544	0.1295	-16%
inertial	6	0.0922	0.1066	+16%
joint	48	0.0854	0.0787	-8%
all 71 together	71	0.0977	0.0888	-9%

Table 7. Identification by parameter group, run C. The average over all 71 parameters is the least informative number in the table: it is dominated by the 48 joint parameters, most of which the data barely constrains.

The two contact-model constants -- the ones ComFree's smooth contact law is built on -- had their error cut in half. The six inertial parameters got worse: the fit trades accuracy in directions the data does not constrain for accuracy in directions it does. That is unidentifiability, not a bug.

Part 6 -- Why the policy does not match the planner

Across every rung the learned policy stays well below the planner. Four explanations are usually offered for this, and they call for different fixes, so they were measured apart rather than guessed between.

Explanation	How it was measured
Mode averaging	MPPI is multi-modal: at one state several unrelated action sequences score alike. A single deterministic policy fitted to all of them lands between them. Measured by comparing the policy's imitation error against the spread inside the elite set itself -- the error no deterministic function can beat.
Mode collapse	The policy emits nearly the same action everywhere. Measured by the standard deviation of its actions over time, against the planner's.
Distribution shift	The policy copies well where the planner looks, then drifts elsewhere when it drives. Measured as the same imitation error on planner-visited and policy-visited states.
A dishonest value function	The policy's objective is mostly 'maximise Q'. Measured by comparing Q's prediction against the discounted return that actually followed.

Diagnosing the distillation gap

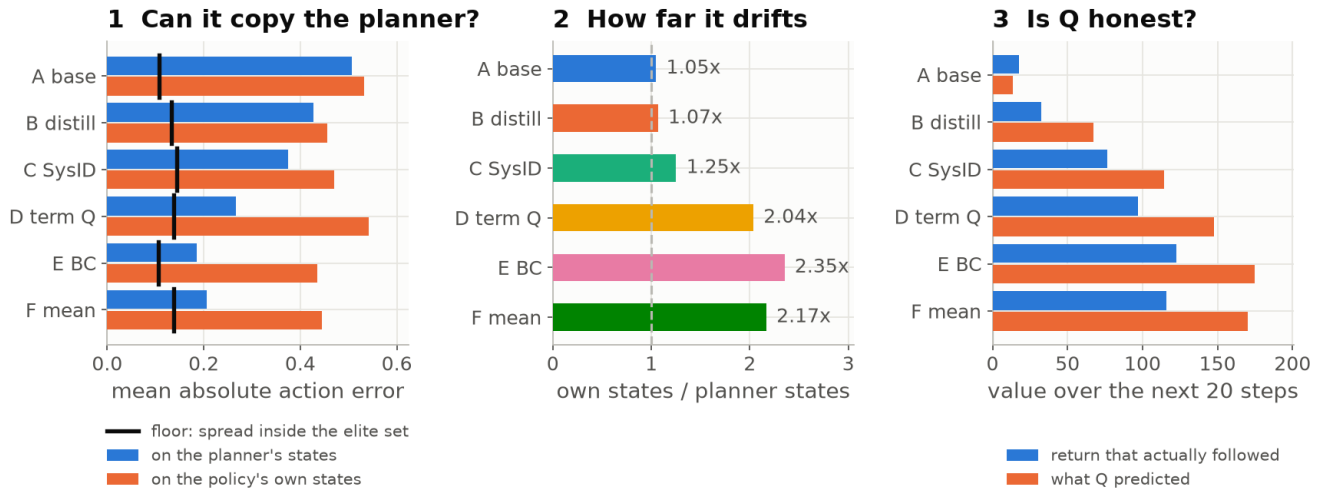


Figure 7. The four measurements, per rung. In panel 1 the black bar is the floor: how far apart the actions the planner itself considers equally good are. Panel 2 is the ratio of the two bars in panel 1.

Run	Copy error (planner states)	Floor	Copy error (own states)	Drift	Q says	Truth	Policy goals
A base	0.507	0.107	0.532	1.05x	14	18	0.57
B + distill	0.427	0.133	0.455	1.07x	67	32	1.14
C + SysID	0.375	0.143	0.469	1.25x	114	77	1.25
D + terminal Q	0.267	0.139	0.542	2.04x	147	97	0.65
E + BC policy	0.185	0.106	0.436	2.35x	175	123	0.65
F + single-mode target	0.206	0.138	0.445	2.17x	170	116	0.93

Table 8. Diagnosis of the distillation gap. Action errors are mean absolute, per action dimension, on a $[-1, 1]$ scale. 'Drift' is the copy error on the policy's own states divided by the copy error on the planner's.

What the measurements rule out

It is not mode averaging. The floor -- how far apart the actions the planner itself scores as equally good are -- sits between 0.11 and 0.14 in every run, while the policy's copy error ranges from 0.51 down to 0.19. The policy is between 1.7 and 4.7 times worse than the best a single deterministic function could do. It never gets close enough to the floor for multi-modality to be what is stopping it.

It is not mode collapse. The policy's actions vary over time by 0.36 to 0.56, against the planner's 0.36 to 0.61. It is not emitting one action everywhere.

It is drift, and the drift gets worse as the planner gets better. Reading down the ladder, the policy copies the planner steadily *better* on the planner's own states -- 0.51, 0.43, 0.38, 0.27, 0.19 -- and yet its measured performance peaks in the middle, at rung C, and then falls. The column that moves with performance is the drift ratio: 1.05, 1.07, 1.25, 2.04, 2.35. At rungs D and E the policy copies the planner twice as badly once it is the one driving.

The best imitator on the ladder (rung E, copy error 0.19, within 1.7x of the floor) is one of the worst controllers. Copying accuracy and control performance come apart, and what separates them is how far the policy's own trajectory leaves the states it was trained on.

This has a plausible reading, though the ablation was not designed to test it and it should be treated as a hypothesis rather than a result. A terminal value makes the planner much more capable -- rung D almost never drops the cube -- and a more capable controller keeps the system inside a narrower, more finely maintained region of state space. Small copying errors then take the policy outside that region faster, where its training data thins out and its errors grow. The better the expert, the less forgiving cloning it becomes.

The value function is optimistic, and increasingly so: Q predicts 68 against a true 33 at rung B, and 175 against 123 at rung E. Its ranking is also weak (correlation with the realised return between 0.23 and 0.59). Since the policy's objective is largely to maximise Q, this is a second real problem -- but it cannot be the main one, because the copy error keeps improving down the ladder while Q gets steadily worse.

Part 7 -- What was learned

1. Throughput was the whole game

The planner is deep and narrow, and a GPU is wide. Mapping environment e , sample i onto world $e \times N + i$ raised the data rate 6.1x on the same card with no change to the algorithm, verified against the simulator's own numerical noise floor. Everything below is only measurable because of it: at the original rate, a single one of these six runs would have taken 4.5 hours instead of one.

2. Distillation works -- once there is enough data

Harvesting the elite MPPI rollouts raised the policy from 0.57 to 1.14 goals per episode. It also made the *planner* better (1.62 to 2.02 goals), which is worth explaining: 24 of the planner's 256 sampled trajectories are generated by the policy instead of by noise. Those 24 are closed-loop -- they react to the state they reach, which an open-loop noise sample cannot -- so a better policy hands the planner a class of candidate it could not otherwise draw.

3. System identification fixes the contact model, not the robot

Averaged over 71 parameters the improvement looks negligible. Split by group it is not: the two contact-model constants had their error halved, contact frictions improved by a sixth, the 48 joint parameters barely moved, and the inertial parameters got worse. The planner nevertheless gained a large amount of reward (89 +/- 28 to 199 +/- 40) without solving more goals -- it made more progress before losing the cube. Predictive accuracy where the data constrains it is what planning needs; recovering the true parameter vector is a harder and different problem.

4. The terminal value transforms the planner

This is the largest single effect in the report. Letting the planner add $\gamma^H Q(s_H)$ to each rollout took it from 1.94 to 4.56 goals per episode, and the fraction of episodes ending with the cube on the floor fell from 0.88 to 0.02.

The mechanism is specific. Dropping the cube costs 50 reward, but a horizon-8 planner only sees that penalty if the drop happens inside those 8 steps. By then it is usually too late -- the cube is already sliding. A learned value function scores the state itself, so 'this grip is heading towards a drop' reaches the planner tens of steps earlier. This is exactly the claim TD-MPC makes for a terminal value, and here it is isolated from everything else.

Terminal Q learning is what this project is for, and it works: 1.94 -> 4.56 goals per episode, drop rate 0.88 -> 0.02.

5. The same Q that saves the planner hurts the policy

The policy moves the other way at rung D: 1.25 down to 0.65 goals. The planner is protected from an over-optimistic Q by eight steps of real simulated reward, which dominate the score. The policy has no such anchor: it maximises Q directly, and it is trained to imitate actions the planner picked partly *because* Q was optimistic about where they led.

Fitting the policy to the elite actions by plain regression (rung E) did not close the gap either: goals stayed at 0.65 while the drop rate fell from 0.51 to 0.20. The policy became more careful, not more capable. Rung F, which hands the policy the plan's weighted mean instead of individual elite actions -- one consistent target per state -- recovered part of it, to 0.93 goals with the policy's episode reward turning positive for the first time. Its error bars overlap rung E's, so this is supporting evidence and not an established result.

That rung was designed to test a hypothesis that the measurements then contradicted. It was built to fix mode averaging, and Part 6 shows mode averaging is not the problem: the elite set is far tighter than the policy's error throughout. Whatever rung F gained, it did not gain by the mechanism it was built for -- more likely the plan mean

is simply a lower-variance regression target than a single sampled elite. Its drift ratio (2.17) does sit slightly below rung E's (2.35), which is consistent with that reading.

What to try next

Idea	Why
Let the policy drive sometimes	This is what the diagnosis points at. The copy error roughly doubles on the policy's own states at the rungs where it performs worst, and that ratio -- not the copy error itself -- is what tracks performance across the ladder. Collecting a fraction of episodes under the policy and having the planner relabel the actions (DAgger) attacks exactly that.
Train the policy without Q	Rung D shows the value function helps the planner and hurts the policy. A policy trained purely by imitation, with Q used only as the planner's terminal value, separates the two roles that are currently in conflict.
Not a richer policy class	Recorded as a negative result: a mixture or diffusion policy is the standard answer to a multi-modal target, and the measurements say that is not what is binding here. The elite set is three to five times tighter than the policy's own error in every run. Reach for this only after the drift is dealt with.
Run longer	The policy curves in Figure 4 are still climbing at 150,000 steps. Nothing here has converged; these are one-hour budgets, not asymptotes.

How much to trust this

Limitation	What it means for the numbers
One seed per rung	Each row of the ladder is a single training run. Differences of the size seen between rungs A, B and C are of the same order as the spread between evaluations, so those three should be read as a trend, not as three separately established results. The rung D effect is roughly ten times the evaluation spread and does not depend on that caveat.
Not converged	150,000 environment steps is one hour of wall clock, chosen so that six configurations fit in a day. The policy is still improving at the end of every run.
Evaluation noise	One evaluation is 16 episodes of a task whose reward is dominated by whether the cube survives. Every number quoted is the mean over the last several evaluations with its spread beside it, never a single final point.
The reality gap is synthetic	'Reality' is the same simulator with 71 displaced parameters, so SysID is being asked to close a gap that is exactly parametric. A real robot differs from its model in ways no parameter vector can express.